Distributed Systems
Homework task #2

Variant 2

Event-Driven Microservices with FastAPI with Message Broker (Python)

Description

The aim of this homework project is to Design and implement **event-driven microservices** using:

1. FastAPI.
2. Asynchronous communication.
3. One of the following message brokers:
    - ActiveMQ
    - RabbitMQ
    - Apache Kafka

The system must follow microservices architecture and communicate asynchronously using events (pub/sub or queues).

Example System Scenario: "Online Order Processing System"

This is a classical example of a distributed backend for a simple e-commerce system. You need to conceive, design, and build **your own** backend system.

Required Microservices

In the Online Orders Processing System you must implement **at least 3 services**:

1. Order Service

- Accepts HTTP requests:
    - `POST /orders`
    - `GET /orders/{id}`
- Saves order to its own database
- Publishes event: `order.created`

---

2. Payment Service

- Listens for: `order.created`
- Simulates payment processing
- Publishes: `payment.completed` OR `payment.failed`

---

3. Notification Service

- Listens for: `payment.completed`, `payment.failed`
- Logs or stores notification
- Simulates sending email/SMS

Communication Requirements

- Services **must NOT call each other directly**
- Communication must be:
  - Asynchronous
  - Event-driven
  - Through broker only

Technical Requirements

Backend

- FastAPI
- Async/await
- Pydantic models

Messaging

Choose one:

Broker	Python Library
ActiveMQ	stomp.py
Kafka	aiokafka
RabbitMQ	aio-pika

Infrastructure

Must use (bonus, for highest grage):

- Docker
- Docker Compose

Each service must:

- Run in its own container
- Have its own database (PostgreSQL or SQLite)
- Connect to message broker container

Possible Project Structure

```
project/
│
├── order-service/
│   ├── main.py
│   ├── models.py
│   ├── publisher.py
│
├── payment-service/
│   ├── main.py
```

```
│   ├── consumer.py
│
├── notification-service/
│   ├── main.py
│   ├── consumer.py
│
├── docker-compose.yml
└── README.md
```

Functional Flow

1. Client creates order
2. Order Service publishes order.created
3. Payment Service consumes event
4. Payment Service publishes result
5. Notification Service consumes result

Bonus Challenges (for high grades)

- Add monitoring (Prometheus)
- Add OpenTelemetry tracing

Possible Alternative Scenarios

- Ride Sharing System
- Food Delivery Platform
- Banking Transaction System
- IoT Event Processing

References

1. FastAPI framework
2. Docker: Accelerated Container Application Development
3. ActiveMQ - Apache Software Foundation
4. Apache Kafka
5. RabbitMQ: One broker to queue them all